“Operating System Design and Implementation”

(Chapter 1)
Operating System Architecture

Er. Bhesh Bahadur Thapa
bhesh.thapa@pu.edu.np

References

Textbook

- ✓ Operating System Concepts, 10th Edition, Abraham Silberschatz
- ✓ Modern Operating System, 3rd Edition, Andrew S. Tanenbaum

Contents

This Chapter will cover:

- Review on operating system: OS concepts, System Calls, Process, Threads, CPU Scheduling, Memory, Storage, I/O Management.
- Operating System Design Goals and Principles
- Operating System Structures and Kernel Types: Layered, Monolithic, Microkernel, Hybrid Kernel, Exokernel, Nanokernel, and Modular Kernel
- Kernel Design and Implementation Strategies
- Case Studies : Linux Kernel Architecture, Windows NT Architecture

Review of Operating System

- An operating system is collection of system programs that control our computer and other peripherals connected to it.
- Basically, there are two views of OS
  - **Top down view:** OS is a program that acts as an intermediary between a user of a computer and the computer hardware, and makes the computer system convenient to use.

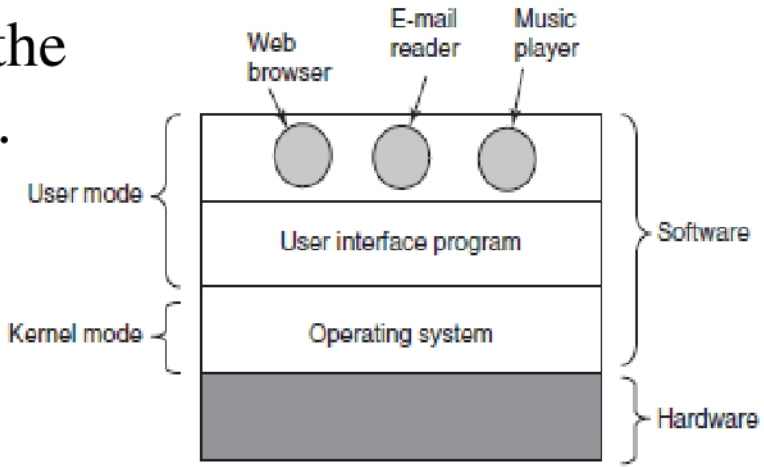


Figure 1-1. Where the operating system fits in.

Review of Operating System…

- **Bottom up view:** Operating system is a resource manager who manages the hardware and software resources in the computer system

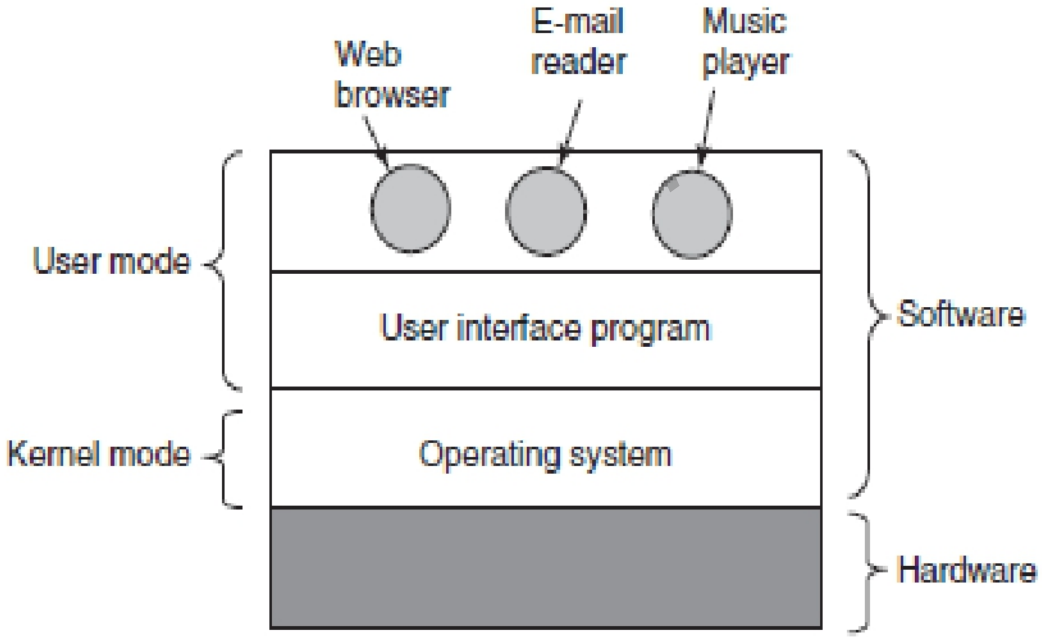


Figure 1-1. Where the operating system fits in.

Review of Operating System…

Function of Operating System

1. Operating System as Resource Manager

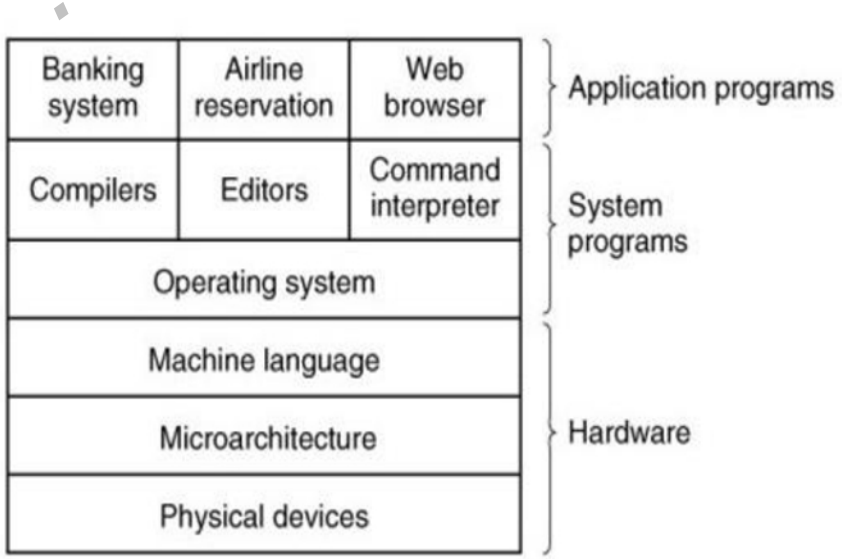
- ✓ Allow multiple programs to run at the same time.
- ✓ Different program and user may try to access common resources simultaneously.
- ✓ Manage and protect memory, I/O devices, and other resources.
- ✓ OS keeps track of who is using which resources, to grant resources requests, to account for usage and to mediate conflicting request from different programs and users.
- ✓ Includes multiplexing (sharing) resources in two different ways:
  1. In time: Printer
  2. In space: Memory

Review of Operating System…

Function of Operating System

1. Operating System as an Extended Machine

- ✓ The architecture of most computers at machine language level is primitive and awkward to program.
- ✓ Operating system programs that act as interface between user and the computer hardware.
- ✓ So operating system sits between the user and the hardware of computer system providing an operational environment.



Review of Operating System…

Function of Operating System

1. Operating System as an Extended Machine/Virtual Machine

Example:

- ✓ Users can use simple commands to perform various tasks and let the operating system do the difficult work of interacting with computer hardware.
- ✓ Thus, we can use a command like copy file1 file2 to copy 'file1' to 'file2' and let the operating system communicate with the controller(s) of the disk that contain(s) the two files.

Operating System Concept: System Call

- System call is an interface between process and the operating system or interface between operating system and its application program.
- Main purpose of system call is to request the operating system to perform some activity i.e. provide some service.
- System calls often use a special machine code instruction which causes the processor to change mode (e.g. to "supervisor mode" or "protected mode").
- This allows the OS to perform restricted actions such as accessing hardware devices or the memory management unit

Operating System Concept: System Call…

Purpose of System Call

- **System call for File manipulation .** For example, create, open, close, read, write etc.
- **System call for Process Management:** There can be system call for process control like abort, execute, terminate, and allocate free memory for a process.
- **System call for device management:** The system call that can be invoked are request device, release device, read, write, get and set attributes.
- **System call for information management:** The System calls helps in making information maintenance such as get/set time
- **System call for IPC (Inter process communication):** System calls are useful for communication purpose as they help in sending or receiving messages, attach or de-attach remote devices, establish and close communication connection etc.

Operating System Concept: System Call…

Example:

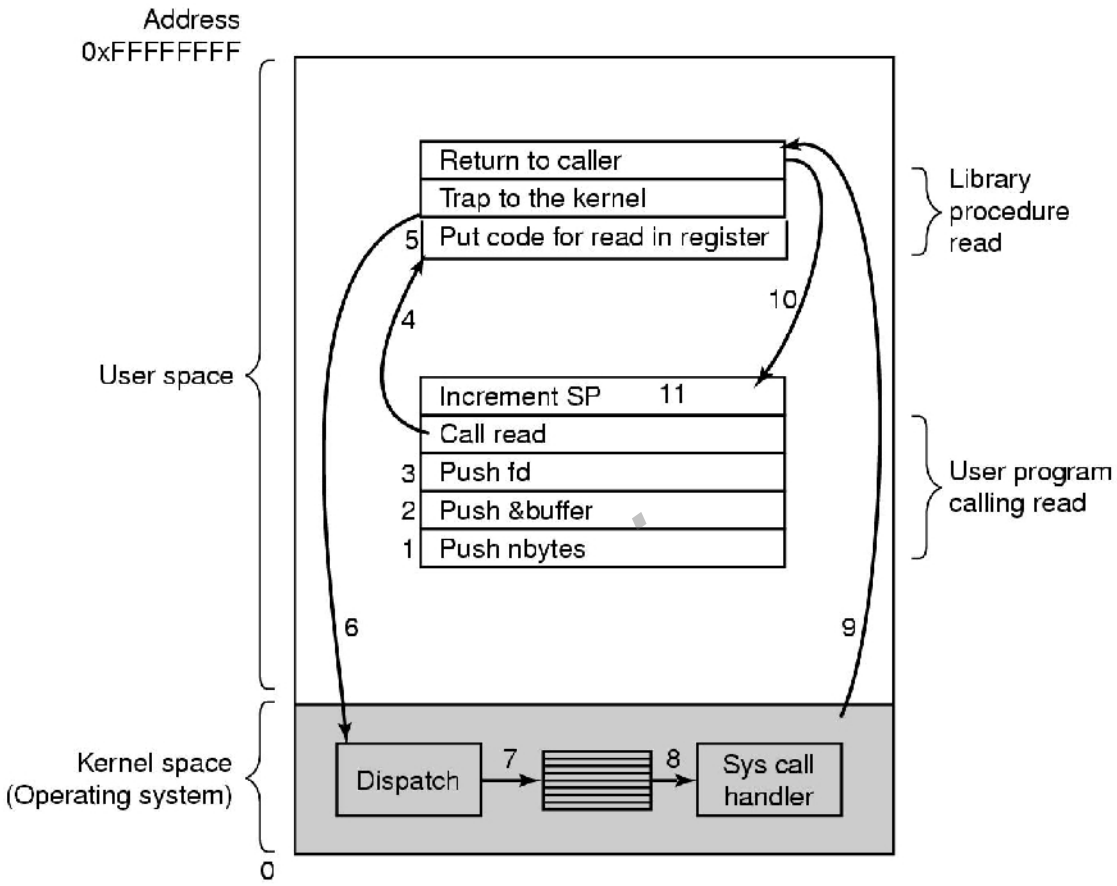


Fig: Steps in Making System Call

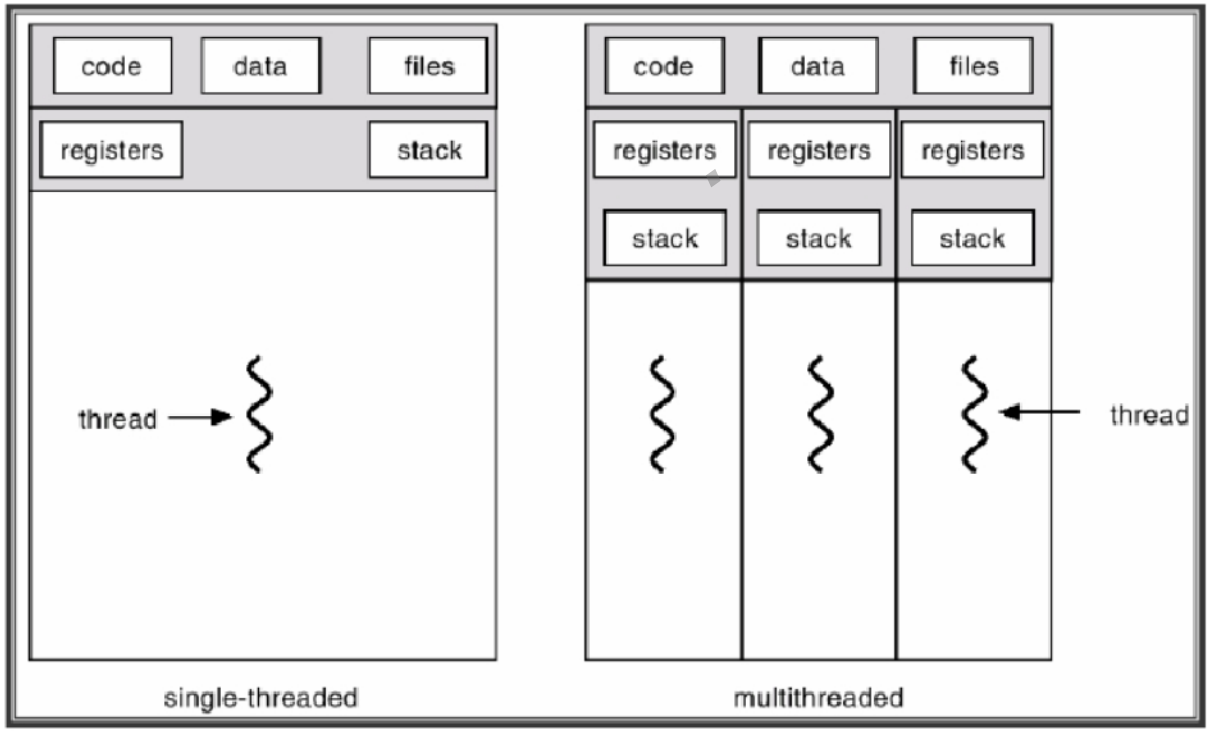
read (fd, buffer, nbytes)

Operating System Concept: Process

- A process is a program in execution. Compilation of an instruction is a process. A task such as sending output to a printer is also a process.
- A process needs some resources including CPU, memory and input/output devices to accomplish the task.
- Every system consists of collection of processes. Some of which are operating system processes (those that executes system codes) and some are user processes (those that executes user code).
- Process is not same as program. Process is an active entity (loaded into main memory) while a program is passive entity. A process is just a part of program.

Operating System Concept: Thread

- A program may consist of multiple independent flow of control called as thread.
- Threads are also called as light weight process because thread run within a program and share resources of that program to which it belongs to.
- Every program has at least one thread which is called as main thread..



4: Threads

Operating System Concept: Thread…

- When a single program is having multiple threads running all at same time and performing different task, then this is known as multithreading.
- Multithreading increases CPU utilization because it allows application to divide its task into individual thread and achieve multitasking.
- The operating system then divide the processing time on these threads similar to the way it divides processing time among the application in multitasking environment.
- Example:
  1. Web browser is an example of multithreaded application. Here user can scroll a page while it is downloading some documents from web server, playing animations and sounds concurrently, printing a page in the background while we are downloading a new web page.
  2. Web Server, which serves multiple clients concurrently accessing it.

Operating System Concept: Thread…

Why Multithreading?

i. **Responsiveness:** Multithreading and interactive application may allow a program to continue running even if part of it is blocked or is performing a lengthy operation, thereby increasing responsiveness to the user.

ii. **Economy:** Allocating memory and resources for process creation is costly. Since threads shares resources of the process to which it belongs, it is more economical to create and context switch threads.

iii. **Resource Sharing:** Threads share the memory and resources of the process to which they belong.

iv. **Utilization of multiprocessor architecture:** Since multiple thread can run parallel on different processor so it provides full utilization of multiprocessor architecture.

Operating System Concept: Thread…

Process	Thread
Process is considered heavy weight	Thread is considered light weight
Unit of Resource Allocation and of protection	Unit of CPU utilization
Process creation is very costly in terms of resources	Thread creation is very economical
Program executing as process are relatively slow	Programs executing using thread are comparatively faster
Process cannot access the memory area belonging to another process	Thread can access the memory area belonging to another thread within the same process
Process switching is time consuming	Thread switching is faster
One Process can contain several threads	One thread can belong to exactly one process

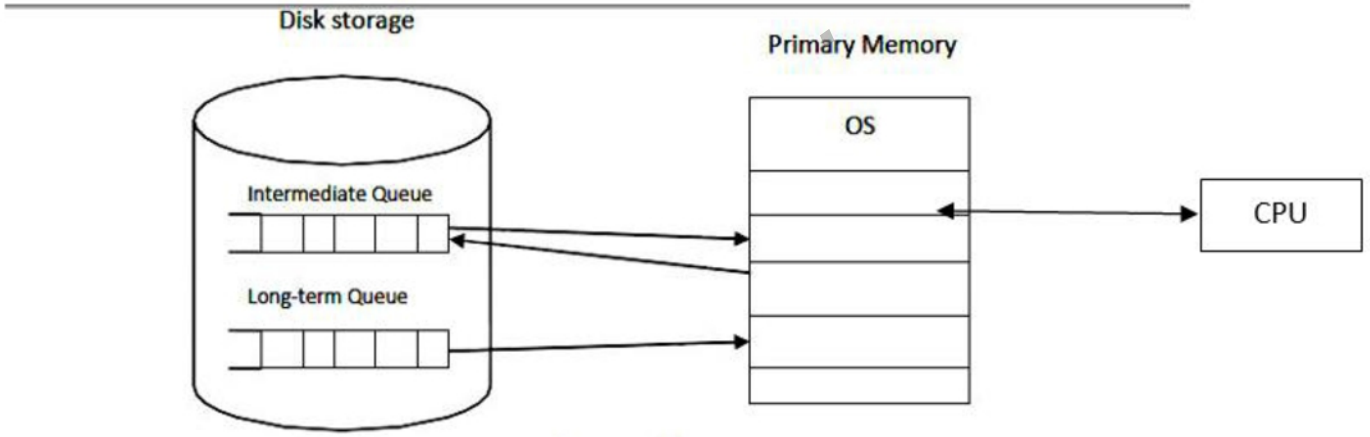
Operating System Concept: CPU Scheduling

- Scheduling is the process of determining which process will actually run when there are multiple runnable processes.
- One of the very important functions of operating system because it can have a big effect on the resource utilization and overall performance of the system.
- This part of operating system is called Scheduler and the algorithm is called as Scheduler algorithm.
- Proper selection of scheduling algorithm is necessary for providing maximum CPU utilization. Some of the selection criteria are
    - ✓ Make sure each process gets equal access to CPU.
    - ✓ Keep the CPU 100% busy.
    - ✓ Minimize the response time for interactive users.
    - ✓ Minimize the time, batch user must wait for I/).
    - ✓ Maximize the number of jobs processed per hour (Throughput).

Operating System Concept: CPU Scheduling

Scheduling Level

1. Long Term Scheduling
2. Middle Term Scheduling
3. Short Term Scheduling



Operating System Concept: CPU Scheduling

Long Term Scheduling

- ✓ The Long Term Scheduler also called as Job Scheduler, selects process from this process pool and loads the selected process into the memory for execution.
- ✓ Once the job scheduler makes a job active, it stays until it terminates.
- ✓ The main objective of long term scheduler is to provide appropriate number of jobs to medium term scheduler i.e. control the degree of multiprogramming.

Operating System Concept: CPU Scheduling

Long Term Scheduling

- ✓ The Long Term Scheduler also called as Job Scheduler, selects process from this process pool and loads the selected process into the memory for execution.
- ✓ Once the job scheduler makes a job active, it stays until it terminates.
- ✓ The main objective of long term scheduler is to provide appropriate number of jobs to medium term scheduler i.e. control the degree of multiprogramming.

Middle Term Scheduling

- ✓ The Medium Term Scheduler also called as swapper, swap the process in and out of memory.
- ✓ It can be sometime needed to reduce the degree of multiprogramming by removing the process from memory and storing them onto the disk.
- ✓ These processes can then be reintroduced into memory by the Medium Term Scheduler.
- ✓ This operation is known as swapping.

Operating System Concept: CPU Scheduling

Short Term Schedule

- ✓ The Short Term Scheduler selects the process to get the processor from among the processes which are already loaded in the memory.
- ✓ Also known as dispatcher, it allocates the processes to the CPU for fixed time quantum and at the end of the quantum the process releases the CPU and returns to the pool of processes from where it was selected.

Operating System Concept: CPU Scheduling

Scheduling Criteria

1. CPU Utilization: Keep CPU 100 percent utilized
2. Throughput: It is the measure of work in term of number of process completed per unit time.
3. Turnaround time: It is defined as the time interval between the time of submission and completion of the job. So it is the sum of the time periods spent in waiting to get into memory, waiting in ready queue, execution on CPU and doing I/O.

Turnaround Time=time of completion of job - tome of submission of job

Or

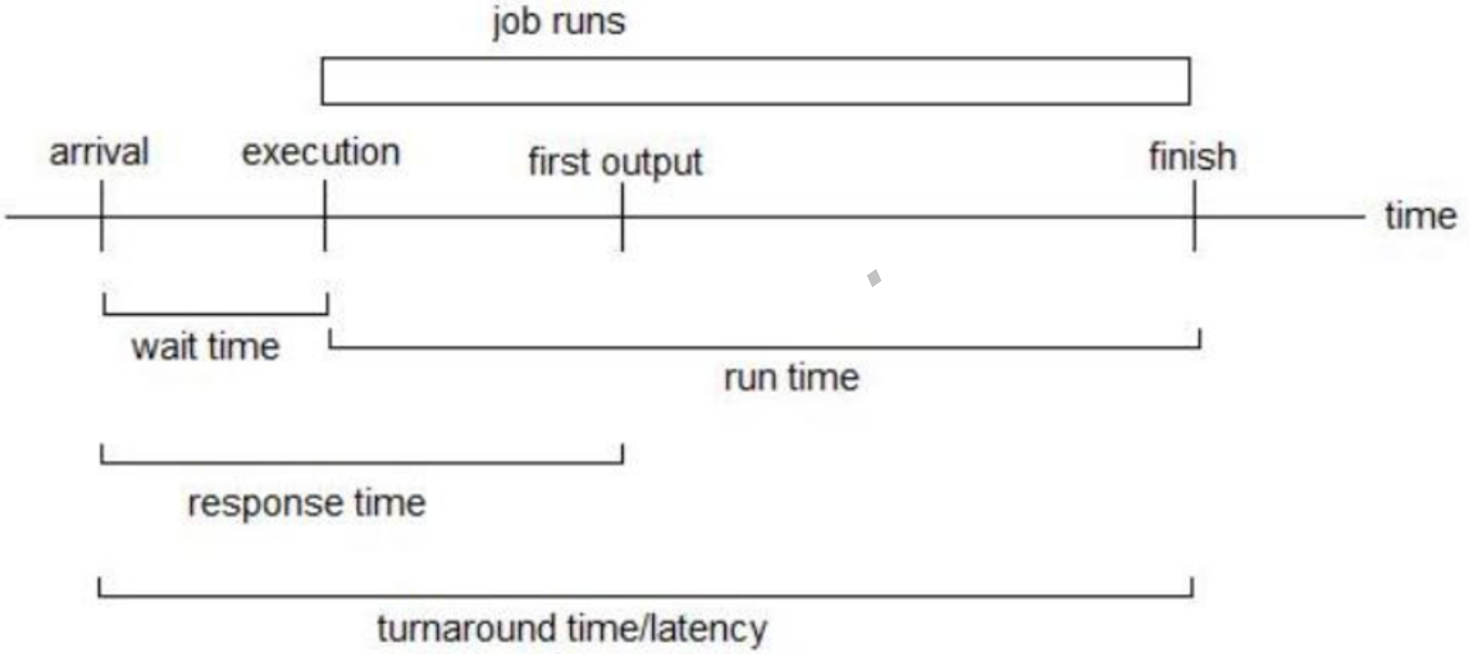
Turnaround Time=waiting time + Burst time or (/) service time

4. Waiting Time: It is the sum of period waiting in the ready Queue.
5. Response Time: It is defined as the time interval between the job submission and the first response produced by the job.

Response time = wait time + Time to first output

Operating System Concept: CPU Scheduling

Scheduling Criteria



Operating System Concept: CPU Scheduling

Types of Scheduling

1. Preemptive Scheduling
2. Non-Preemptive Scheduling

- ✓ In **Preemptive Scheduling**, even if the CPU has been allocated to a certain process, it can be snatched by any other process at any time either due to time constrain or due to priority reason.
- ✓ In **Non-Preemptive Scheduling**, once the CPU is allocated to a process, the process keeps the CPU until it terminates or its transition to blocked state. This means that once a CPU is allocated to a process, this process can use the CPU for its own execution till it willingly surrenders or leave the CPU.

Operating System Concept: CPU Scheduling

Scheduling Algorithm

1. **First Come First Service (FCFS)**
   - FCFS is a simple **non-preemptive scheduling algorithm.**
   - Processes are assigned to the CPU in the order in which the process arrive

2. **Priority**
   - In this algorithm, each process in the system is given a priority, then the scheduling must be done according to the priority of each process i.e. CPU is allocated to the process with the highest priority.
   - Priority scheduling can be both **preemptive and non-preemptive** type.
   - Has Starvation issues.
   - If same priority then decision is based on shortest job length. Again if same then decision based on FCFS.

Operating System Concept: CPU Scheduling

Scheduling Algorithm

3. Round Robin(RR)

- The basic purpose of this algorithm is to support time sharing system.
- RR scheduling is preemptive at the end of time slice therefore it is effective in time sharing environment in which the system needs to guarantee reasonable response times for interactive users.
- This algorithm is similar to the FCFS but now it is preempted FCFS algorithm.

4. Shortest Job First(SJF)

- Key concept of this algorithm is "CPU is allocated to the process with Least CPU Burst Time".
- If the two processes having the **same length**, next CPU burst time, **FCFS** is used i.e. one which arrives first, will be taken up first by the CPU.
- Non Preemptive by default.

Operating System Concept: CPU Scheduling

Scheduling Algorithm

5. Shortest Job First(Preemptive)| Shortest Remaining Time First(SRTF)

- SRT is the preemptive counter part of SJF and is useful in time sharing system.
- In SRT, process with smallest estimated run time to completion is run next.
- In SJF once a job begins executing, it runs to completion. In SRT, a running process may be preempted by a new process with a shorter estimated run time.

6. Highest Response Ration Next (HRRN):

- **C**orrects the weakness of SRTN policy i.e. excessive discrimination towards the longer jobs and excessive favoritism towards the shorter job.
- HRN is **non-preemptive** scheduling algorithm in which the priority of each job is a function not only of job's service time but also of the amount of time the job has been waiting for service.
- HRN derive a dynamic property value based on the estimated service time and waiting time. The priority for each process is calculated from

  Priority P = (Waiting time + service time) / service time

Operating System Concept: CPU Scheduling

1. Consider a set of processes P1, P2, and P3 and their CPU burst time, priorities and arrived time being mentioned as below

Process	CPU Burst Time	Arrival Time	Priority
P1	4	0	2
P2	6	1	3
P3	2	2	1

Assuming 1 to be the highest priority calculate

i. Average waiting time using the FCFS, SJF (Preemptive and Non-Preemptive) and priority (Preemptive and Non Preemptive) ,Preemptive FCFS (RR) and HRRN.

ii. Average turnaround time using all the above scheduling mechanism.

iii. Assume time quantum to be 2 unit of time.

Operating System Concept: CPU Scheduling

2. Consider a set of processes P1, P2, and P3 and their CPU burst time, priorities and arrived time being mentioned as below

Process	CPU Burst Time	Arrival Time	Priority
P1	5	2	1
P2	4	3	1
P3	10	7	2
P4	4	4	1

Assuming 1 to be the highest priority calculate

i. Average waiting time using the FCFS, SJF (Preemptive and Non-Preemptive) and priority (Preemptive and Non Preemptive) ,Preemptive FCFS (RR) and HRRN.

ii. Average turnaround time using all the above scheduling mechanism.

iii. Assume time quantum to be 2 unit of time.

Operating System Concept: Memory Management

- Memory is one of the important resources that must be carefully managed.
- Most computer have a memory hierarchy, with a small amount of very fast, expensive, volatile **Cache Memory**, some numbers of megabytes of medium speed, medium price, volatile memory (**RAM),** hundreds or thousands of megabytes of slow, cheap, nonvolatile **Disk** storage.
- The part of the Operating system that manages the memory hierarchy is called **Memory Manager.**
- Its job is to keep track of which part of memory are in use and which part are not in use, to allocate memory to process when they need it and de-allocate it when they are done, memory protection, and to manage swapping between main memory and disk when main memory is not big enough to hold all the processes.

Operating System Concept: Memory Management

Storage Allocation

- Storage allocation is the assignment of particular areas of a magnetic disk/ Memory to particular data or instructions.
- An allocation method refers to how disk / Memory block are allocated for program data and instruction.
- Storage allocation can be of two types:
    i. Contiguous storage allocation
        - Classical allocation model in which a program data and instruction are assured to occupy a single contiguous memory area.
        - Contiguous storage allocation can be further divided into
            1. Fixed(Static) Partition storage allocation
            2. Variable(Dynamic) Partition storage allocation.
    ii. Noncontiguous storage allocation

Operating System Concept: Memory Management

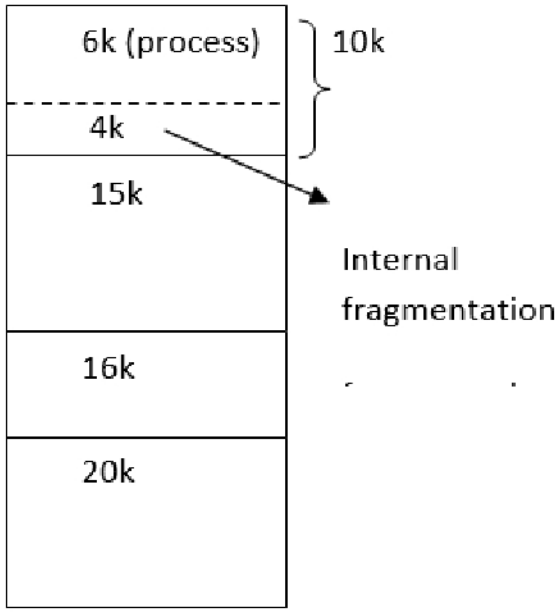
Fixed (static) Partition Storage Allocation

•One of the simple methods of memory allocation is to divide memory into several fixed partition. Each partition can hold one process.

•Fixed partitions can be **equal or unequal.**

•Consider that the primary memory is statically partitioned into N fixed partitions or regions, where region Ri has Ni unit of memory. Typically, Ni and Nj are of different size, so processes with small space use small partition and processes with large space will use large partition. This is known as fixed partition contiguous allocation. If a process requires N_k unit of memory, it can be loaded into any region of partition Ri where $Ni > N_k$ and $Ni - N_k$ unit of the primary memory are left unused.

•This phenomenon where part of space allocated to the process is left unused is called **Internal fragmentation.**



Operating System Concept: Memory Management

Variable (dynamic) Partition Storage Allocation

- Whenever a process needs memory a search for the space (holes) needed by it is done.
- If the contiguous space is available to accommodate that process, then the process is loaded into the memory.
- A hole is a block of available memory which is scattered throughout the memory.
- When a process arrives, it is allocated memory from a hole large enough to accommodate it i.e. When a process is brought into memory, it is allocated exactly as much memory as it requires and no more
- Some process after finishing their execution will leave the memory and new may enter into it.
- This phenomenon of entering and leaving the memory can cause the formation of unusable memory holes. This is known as **External Fragmentation.**

Operating System Concept: Memory Management

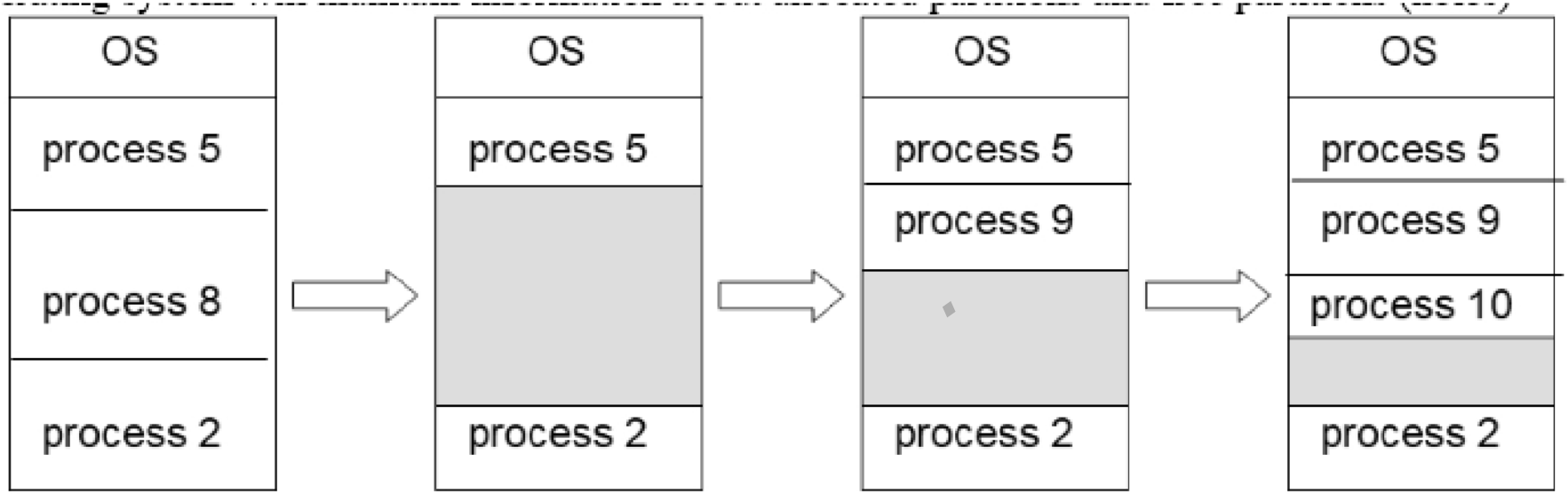
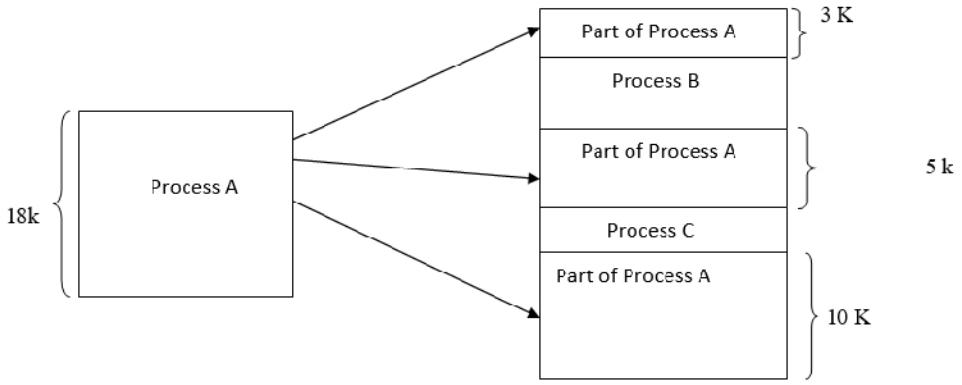


Fig: Variable Partition Contiguous Storage Allocation

Operating System Concept: Memory Management

Non Contiguous Storage Allocations

- To **resolve the problem of external fragmentation** and to enhance the degree of multiprogramming to a greater extent, it was decided to sacrifice the simplicity of allocating contiguous memory to every process and have a non-contiguous physical address space of a process so that process could be allocated memory whenever it was available.
- In this storage allocation technique, the memory used for storing the user program **can be scattered.**
- **Paging and Segmentation** are the memory management techniques that permits the physical address space of process to be non-contiguous.



Operating System Concept: Memory Management

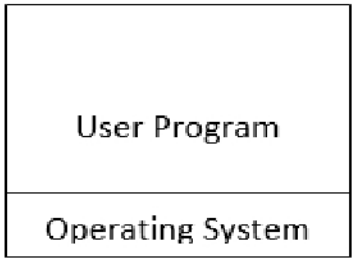
Basic Memory Management

- Mono Programming
- Multiprogramming with Fixed Partition
- Swapping
- Multiprogramming with Variable Partition
- Memory Management with Bitmap
- Memory Management with Linked List

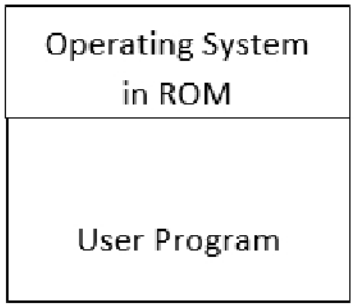
Operating System Concept: Memory Management

1. Mono-Programming without swapping or paging

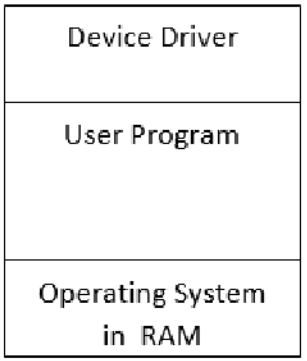
- This is the simplest memory management technique in which just one program is run at a time. Used in MS-DOS System
- As soon as the user types the command, the operating system copies the requested program from the disk to the memory and executes it.
- The memory is shared between that program and Operating system.
- Three variations on this theme are as below.
  i. The operating system may be at the bottom of memory in RAM.
  ii. The operating system may be in the ROM at the top of memory.
  iii. The device driver may be at the top of the memory in a ROM and the rest of the system in RAM.



Variant-I



Variant-II



Variant-III

Operating System Concept: Memory Management

2. Multiprogramming with fixed partition:

- On time sharing system, having multiple processes in memory at once allows another process to use the CPU if one process is blocked waiting for I/O to finish, thus increases the CPU utilization.
- The easiest way to achieve multiprogramming is simply to divide the main memory into number of static partitions at system generation time.
- A process may be loaded into a partition of equal or greater size.
- Memory manager will allocate a partition to a process that best fits it.
- Unused memory within the allocated partition is **called internal fragmentation.**
- Partition can be **equal or unequal**.
- In the case of equal partition, any process whose size is less than or equal to the partition size can be loaded into any available partition.
- In the case of unequal partition, each process is assigned to the smallest partition within which it will fit. The advantage of this approach is that processes are always assigned in such a way as to minimize wasted memory within a partition.

Operating System Concept: Memory Management

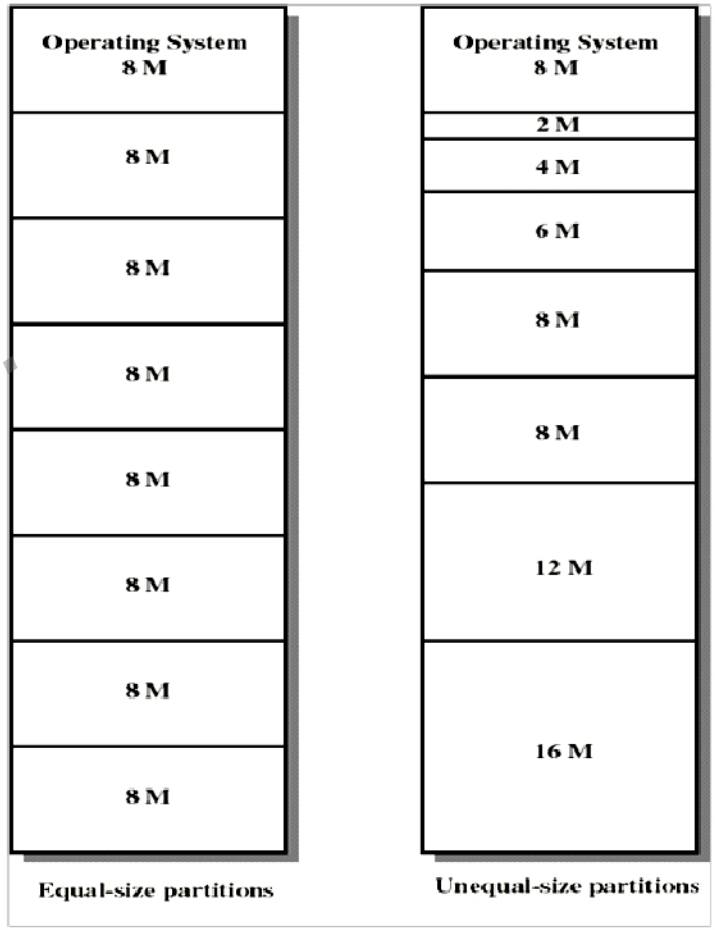
2. Multiprogramming with fixed partition

Advantage

- Simple to implement
- Little OS overhead

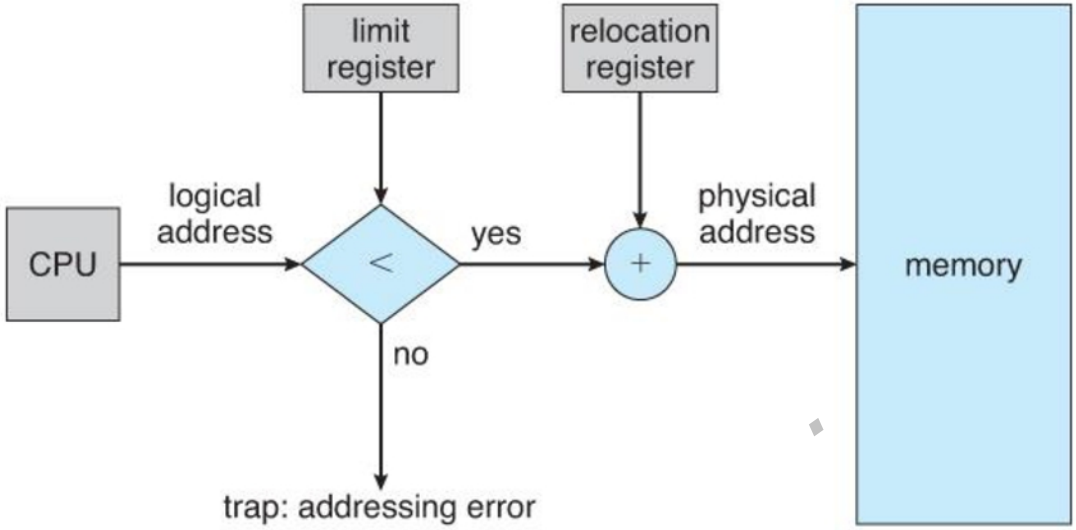
Disadvantage:

- Inefficient use of memory due to **internal fragmentation**. This phenomenon, in which there is wasted space internal to a partition due to the fact that the block of data loaded is smaller than the partition, is referred to as **internal fragmentation**.
- A program may be too big to fit into partition.
- Multiprogramming introduces overhead of **relocation and protection** mechanism**.**



Operating System Concept: Memory Management

Relocation and Protection



- In multiprogramming environment, several user processes to reside in the memory at the same time. Different process run at different address space.
- So protection deals with following issues.
    i. Protection the operating system from user processes.
    ii. Protecting the user processes from one another.
- We can provide this protection by using a relocation register(base) and limit register.

Operating System Concept: Memory Management

Relocation and Protection

- The relocation register contains the value of small physical address (base address) while the limit register contains the length of partition.
- With relocation and limit registers, each logical address must be less than the limit register otherwise it will generate addressing error.
- When any process is scheduled, the relocation register or base register is loaded with the address of the start of its partition, and the limit register is loaded with the length of the partition where that process is stored. Every memory address generated automatically has the base register contents added to it before being sent to main memory.
- Thus if the base register is 200k, a CALL 100, instruction is effectively turned into a CALL 200+100 instruction, without the instruction itself being modified. Addresses are also checked against the limit register to make sure that they do not attempt to address memory outside the current partition.
- The MMU maps the logical address dynamically by adding the value in relocation register.
- Finally, the mapped address is sent to memory

Operating System Concept: Memory Management

Allocation (Placement) Algorithms:

- Different allocation algorithms are used to determine which partition to allocate to a process. Some of them are

1. **First Fit**
   - In this method, search for the hole starts at first block and ends as soon as the block large enough to hold the process requirement is found.
   - Faster in making allocation of memory space.
2. **Next Fit**
   - It works similar to First Fit except that it starts searching the list from the place where it left off last time, instead of always at the beginning as First Fit does.
3. **Best Fit**
   - In this method, the entire available memory blocks are searched; the block that is best suited to the requirement is selected.
   - Internal fragmentation reduced but is not completely eliminated.
4. **Worst Fit**
   - This method always takes the block with largest available hole thus provides maximum internal fragmentation

Operating System Concept: Memory Management

- Given hole size 21k, 10k, 41k, 36k, 16k, 20k, 26k, 32k. Which hole is taken for successive segment of 26k, 22k, and 18k. Which algorithm is efficient? Comment your answer.
- For the following partition of 100k, 500k, 200k, 300k and 600k in order, place the process 212k, 417k, 122k and 426k in order according to first, next best and worst fit.

Operating System Concept: Memory Management

3. Swapping

- A process must be in memory to be executed.
- A process, however can be swapped temporarily out of memory to backing store and then brought back into memory for continued execution.
- For example, assume a multiprogramming environment with round robin CPU scheduling algorithms. When a quantum expires, the memory manager will start to swap out the processes that just finished and to swap another process into the memory space that has been freed.

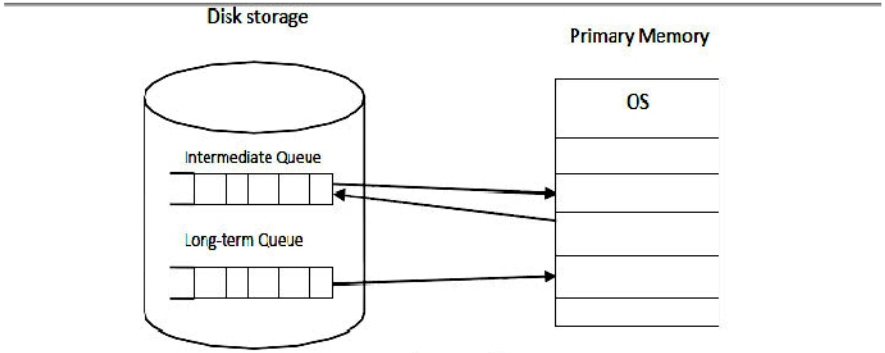


Fig: the use of swapping

The operation of a swapping system is illustrated below

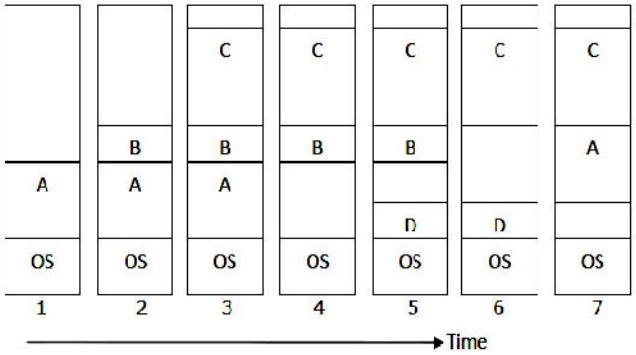
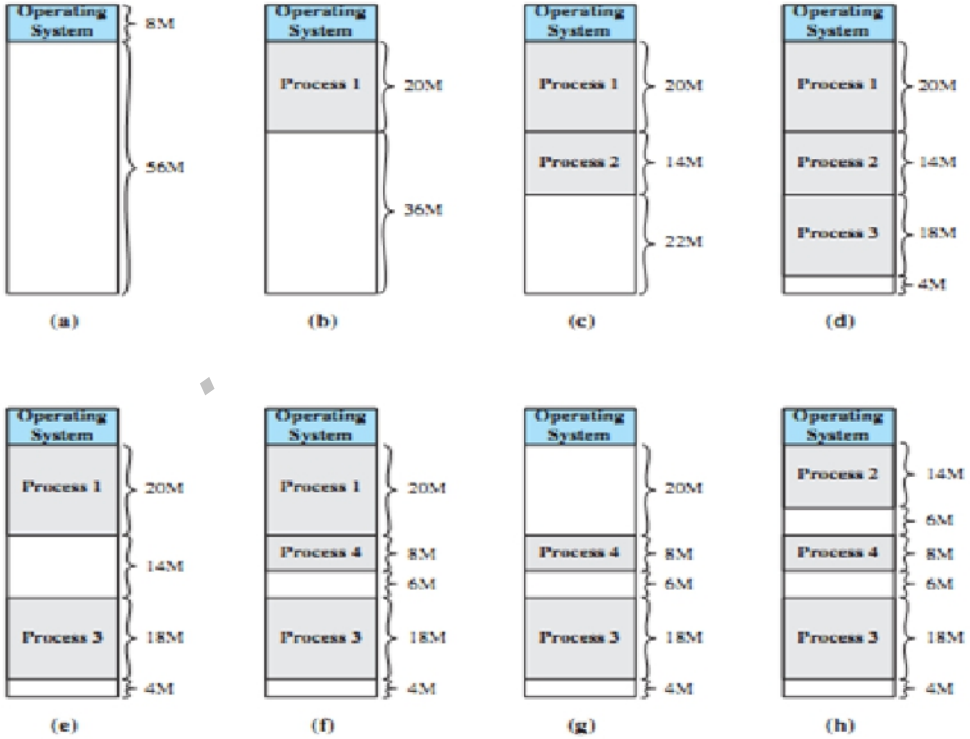


Fig: Illustration for swapping process.

Operating System Concept: Memory Management

4. Multiprogramming with Variable Partition

- With this technique, the partitions are of variable length and numbers.
- When a process is brought into memory, it is allocated exactly as much memory as it requires and no more.
- The operation of swapping system is illustrated in aside figure.
- So the dynamic partition will eventually lead to a situation where there are a lot of small unused holes in memory which declines them memory utilization i.e external fragmentation



Operating System Concept: Memory Management

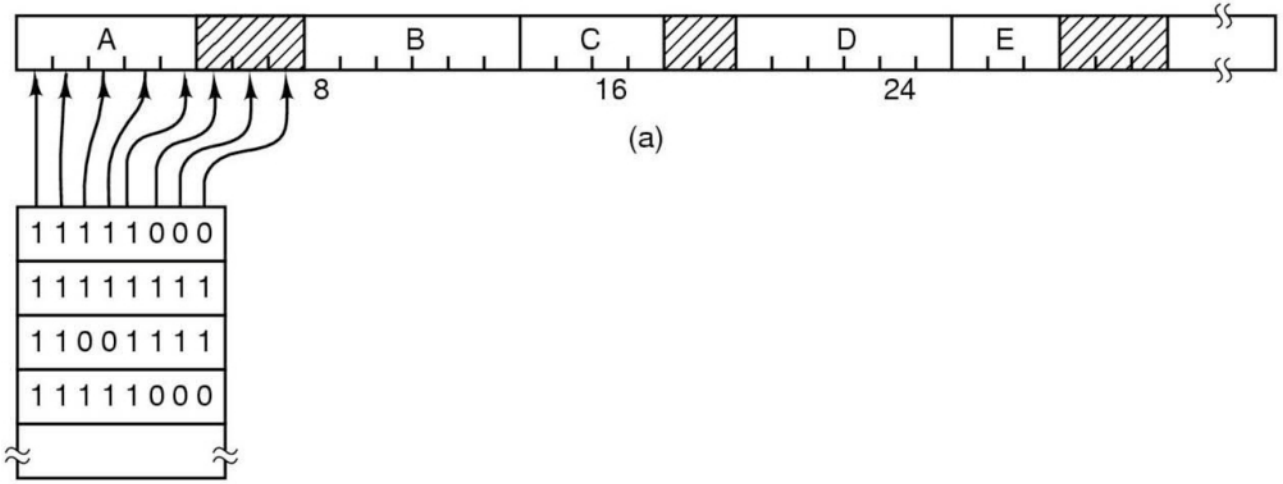
4. Multiprogramming with Variable Partition

- One technique for overcoming **the External Fragmentation is compaction**.
- From time to time, the operating system shifts the processes so they are contiguous and so that the entire unused memory hole is together in one block.
- Also the overhead of relocation and protection still exist.

Operating System Concept: Memory Management

5. Memory Management with Bitmap

- When a memory is dynamically allocated, it's the task of operating system to keep track of memory usage i.e. which part is used and which is free. **Bitmap is one of the techniques to track the memory usage.**
- With a bitmap, memory is divided into allocation units that may range from few **words to large kilobytes**.
- Corresponding to each allocation unit is a bit in the bitmap, which is 0 if the unit is free and 1 if it is occupied.
- Figure shows the part of memory and the corresponding bitmap



Operating System Concept: Memory Management

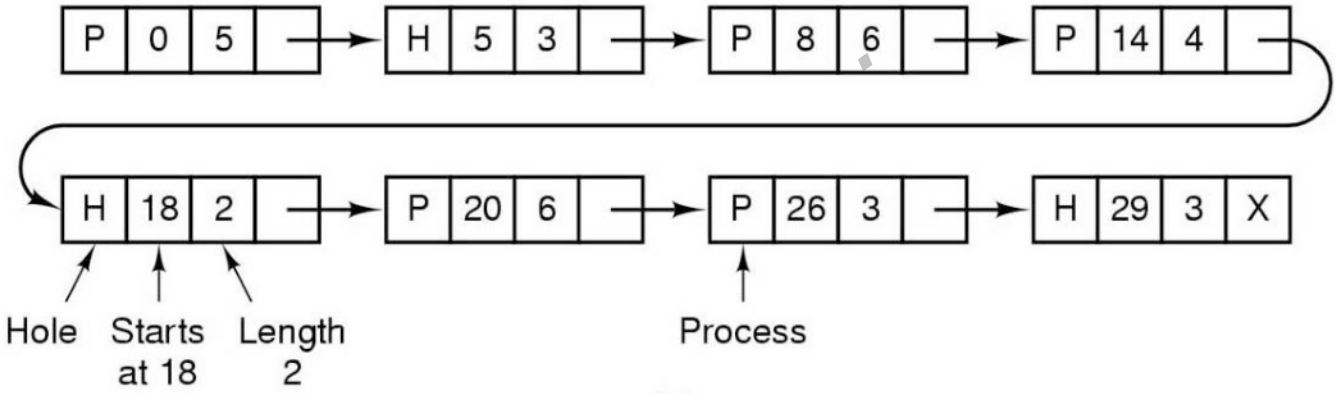
5. Memory Management with Bitmap

- The size of allocation unit is important in design issue. The smaller the allocation unit, the larger the bit map.
- If the allocation unit is larger, the bitmap will be smaller, but may leads to problem of internal fragmentation if the process size is not an exact multiple of allocation unit.
- The main problem with this technique is whenever any k unit or process has to be brought into memory, the memory manager must search the bit map to find a run of k consecutive bit in the map.

Operating System Concept: Memory Management

6. Memory Management with Linked List

- Similar to bit map, linked list is also a technique to keep track of memory usage.
- In this technique, a linked list of allocated and free memory segment is maintained, where a segment is either a process or a hole between two processes.
- The memory of above figure (that in bitmap) is represented as a link list of segment as below.



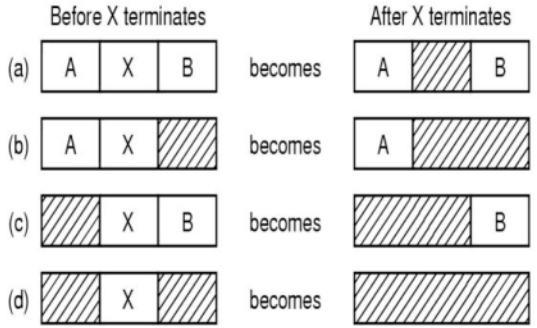
(c)

- Each entry in the list specifies a hole (H) or process (P), the address at which it starts, the length, and the pointer to the next entry.

Operating System Concept: Memory Management

6. Memory Management with Linked List

- A terminating process has normally two neighbors (except when it is at very bottom or top of memory).



- In figure a, when the process x terminates creating hole, the list should be updated by replacing the process (P) by hole (H).
- In figure b, when the process x terminates creating hole, the list should be updated by **Coalescing** (merging of two entries) both the newly created hole and neighbor hole to right. So the list becomes one entry shorter.
- In figure c, when the process x terminates creating hole, the list should be updated by coalescing both the newly created hole and neighbor hole to left. So the list becomes one entry shorter.
- In figure d, when the process x terminates creating hole, all three entries are merged and two items are removed from the list.

Operating System Concept: I/O Management

- The two major jobs of computer are **I/O and processing.**
- In most cases the main job is I/O. For example, when we browse a webpage or edit a file, we are interested to read or enter some information, not to compute the data.
- The role of operating system in computer I/O is to **manage and control I/O operation and I/O devices** i.e. control of devices connected to the computer is the major role of operation system.
- OS should issue **commands to the I/O devices, catch interrupt and handle errors**. It also provides an **interface between the devices and the rest of the system that is simple and easy to use**. To the extent possible, the interface should be the same for all devices (device independence)
- Since I/O devices varies so widely in both **function and speed**, a variety of methods are needed to control them.
- To control these devices, **drivers are installed and are present in the kernel** of operation system.

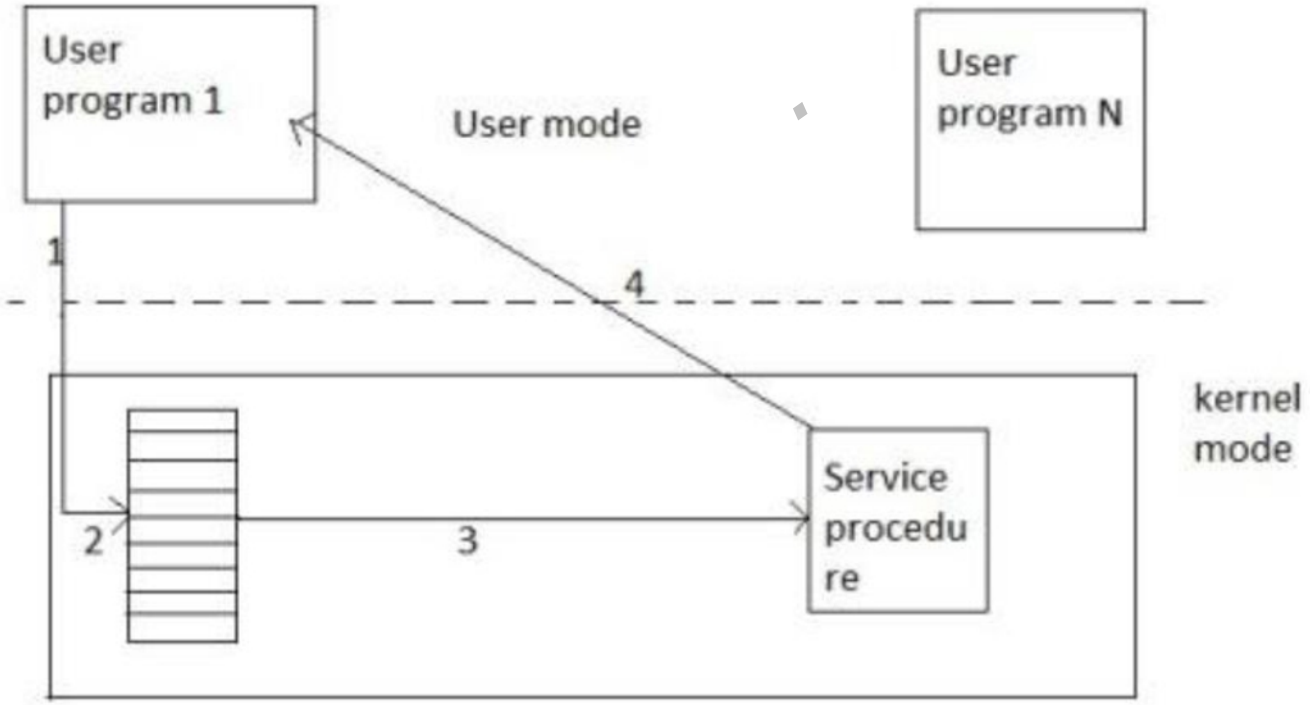
Operating System Structures and Kernel Types

- Modern operating systems are designed by dividing the system into smaller, well-defined modules with specific interfaces and functions.
- A system as large and complex as a modern operating system must be engineered carefully if it is to function properly and be modified easily
- Example: Similar principles apply to structuring programs—organizing logic into separate functions with clear parameters and return values, instead of putting all code in the main function.
- The major architectures/structures of operating systems include the following:
    i. Monolithic
    ii. Layered
    iii. Microkernel
    iv. Exokernel, Nanokernel, and Modular Kernel
    v. Hybrid Kernel

Operating System Structures and Kernel Types

Monolithic Structure of Operating System

- It is the oldest architecture used for developing operating systems.
- The operating system resides on the kernel.
- For any command to be executed, a system call is invoked, switching from user mode to kernel mode and transferring control to the operating system.
- Example Traditional UNIX, MS-DOS.



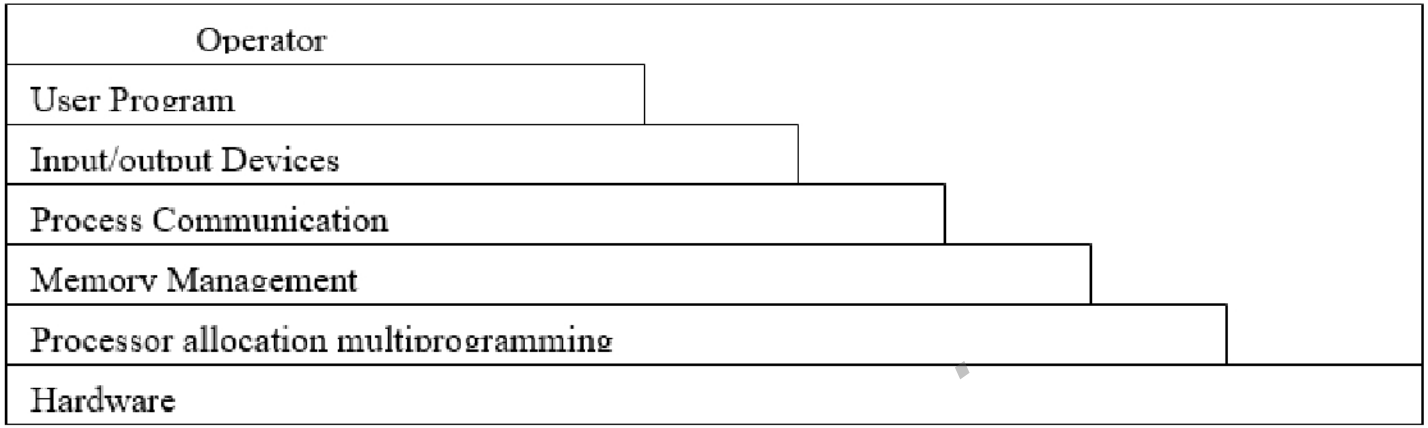
Operating System Structures and Kernel Types

Layered Structure of Operating System

- The layered architecture for operating systems was developed in the 1960s, dividing the system into multiple layers, each built upon the one below it.
- The hardware layer forms the bottom (layer 0), while the user interface (UI) layer resides at the top (layer 6).
- Each layer uses only the functions and devices of the lower-level layers, allowing independent debugging of layers.
- Debugging starts with the bottom layer, and once it is verified, higher layers are debugged sequentially, simplifying error detection.
- The primary drawback is the need for precise layer definitions and careful planning to ensure the proper placement of functions across layers.

Operating System Structures and Kernel Types

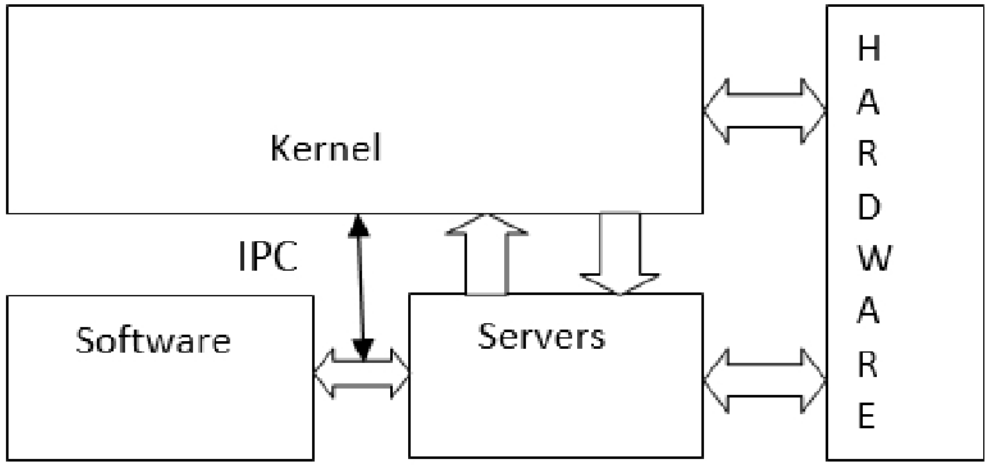
Layered Structure of Operating System



Operating System Structures and Kernel Types

Microkernel Structure of Operating System

- A micro kernel tries to run most services like file system, thread system, etc. as server in user space.
- So, this architecture takes out maximum functionality from kernel as far as possible.
- All that left to do for the kernel are basic services like memory management, scheduling etc.
- This architecture controls the problem of ever-growing size of kernel which could not be controlled in monolithic approach.
- In this architecture, all the basic OS services which are made part of user space are made to run as servers which are used by programs in the system through inter process communication (IPC).



Operating System Structures and Kernel Types

Microkernel Structure of Operating System

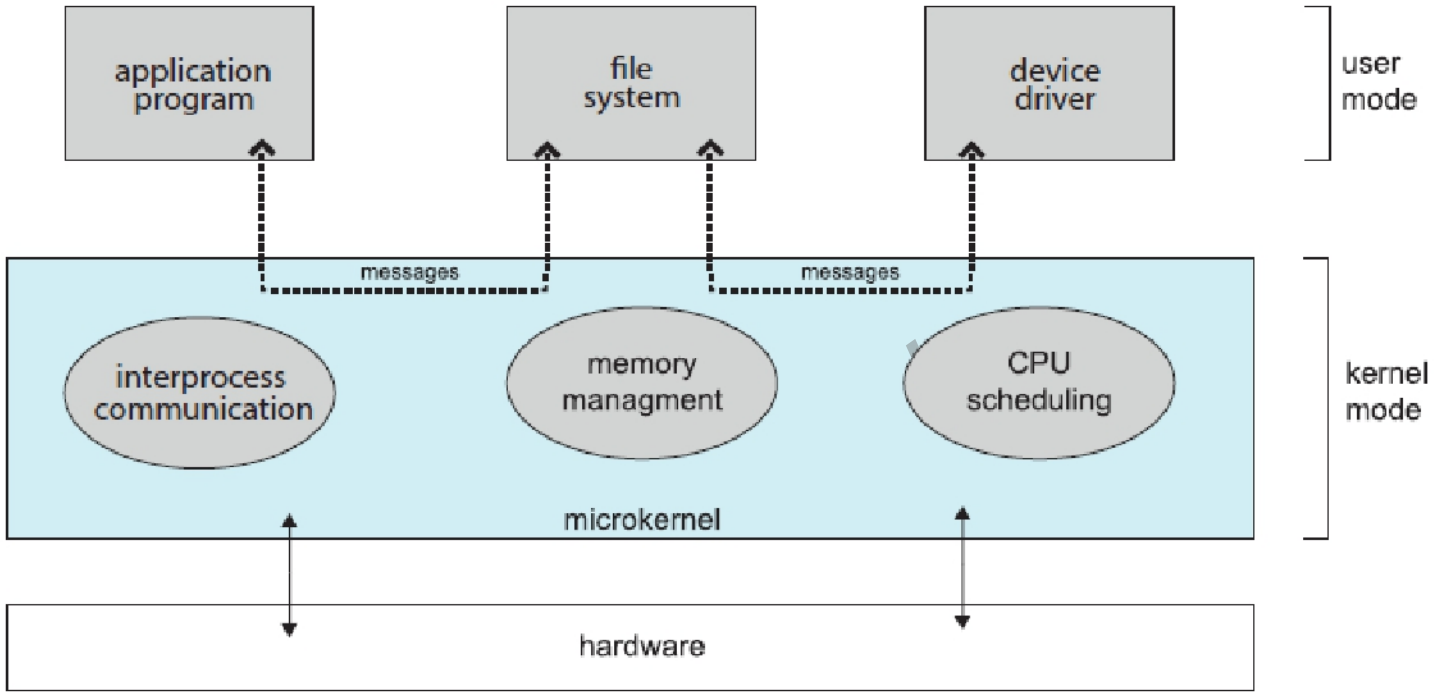
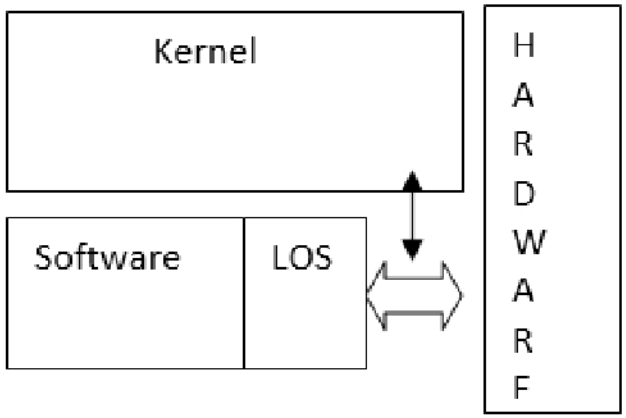


Figure 2.15 Architecture of a typical microkernel.

Operating System Structures and Kernel Types

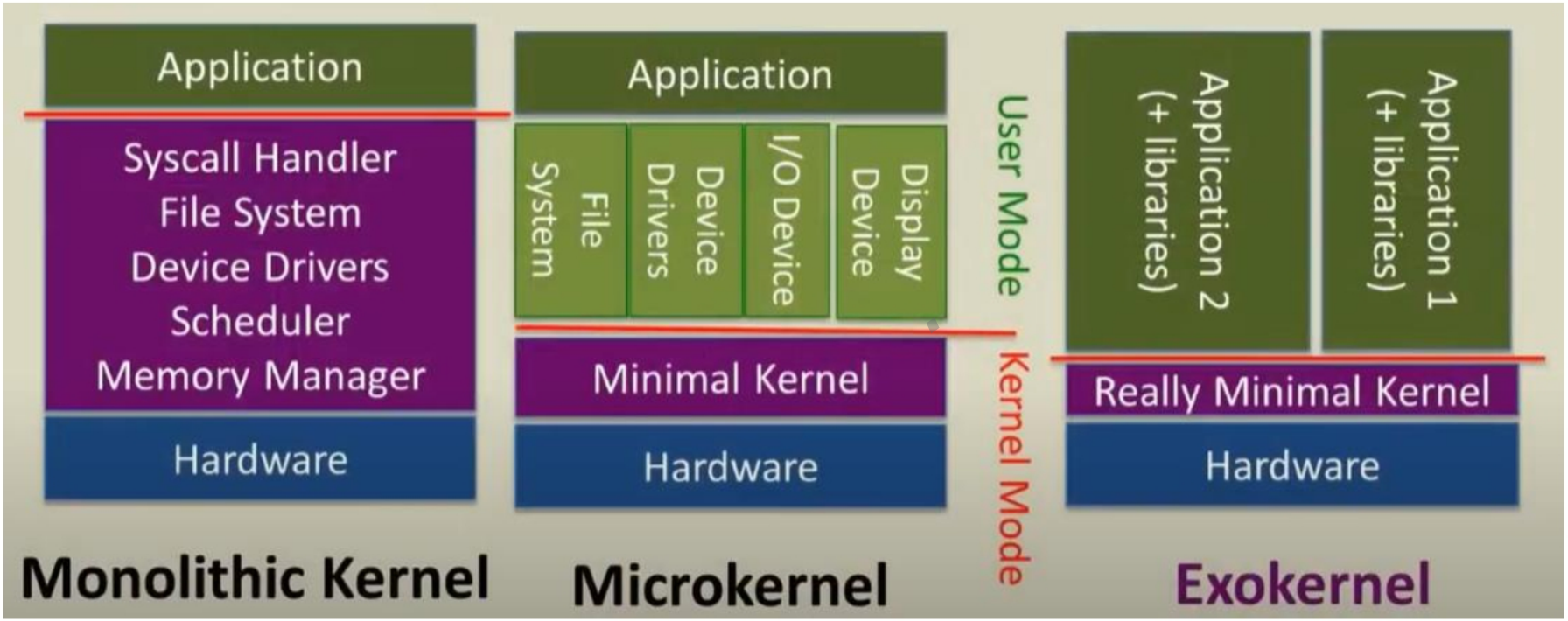
Exokernel Structure of Operating System

- Exo-Kernel is a further extension of the micro-kernel approach where the kernel is almost devoid of functionality. It is still experimental approach to OS design.
- Exokernels reduce system-defined abstractions, letting developers decide hardware abstractions.
- The kernel handles only security and resource multiplexing, delegating tasks to LibOSes.
- LibOSes enable custom abstractions, reducing overhead and enhancing performance.
- Exokernels ensure low-level hardware access, allowing flexible abstraction choices.



Operating System Structures and Kernel Types

Monolithic Vs Microkernel Vs Exokernel



Operating System Structures and Kernel TypesI/O Management

Nanokernel Structure of Operating System

- A nanokernel is even more minimalistic than a microkernel, providing only the most essential functions like interrupt handling, hardware abstraction, and basic resource management.
- It delegates almost all other tasks (such as memory management, process scheduling, and file systems) to higher layers, often in user space.
- The system structure is designed to be as small and simple as possible, with a very limited scope of responsibilities. Most tasks are pushed to user space or higher-level modules.
- Nanokernels are typically designed for real-time or embedded systems, where the focus is on providing a small, fast, and predictable kernel with minimal overhead. They prioritize deterministic performance.

Operating System Structures and Kernel Types

Modular Structure of Operating System

- Perhaps the best current methodology for operating system design. Here, the kernel has a set of core components and can link in additional services via modules, either at boot time or during run time. This organizes an operating system into smaller, self-contained modules.
- The system is divided into independent modules (e.g., memory management, file system, process scheduling, device drivers).
- Each module is responsible for a specific aspect of the system's operation but work together to provide the full functionality of the system.
- Communicates with other modules using interfaces, allowing for flexibility and extensibility.
- Unlike Layered architecture which is ideal for operating systems where simplicity, clear separation of concerns, and ease of debugging are priorities, Modular architecture is often used for systems that require flexibility, scalability, and extensibility.
- Example: Linux, Windows NT Kernel

Operating System Structures and Kernel Types

Hybrid Structure of Operating System

- Most operating systems combine multiple architectural structures to balance performance, security, and usability, rather than adhering to a single strict structure.
- Linux, primarily monolithic for performance, as it operates within a single address space, but also modular, allowing dynamic addition of kernel functionalities.
- Windows, largely monolithic for efficiency but incorporates microkernel-like behaviour, supporting user-mode subsystems (operating-system personalities) and dynamically loadable kernel modules.
- Example: Apple macOS, iOS, and Android are examples of hybrid systems that blend multiple structural approaches.

Further Studies: Detailed case studies of Linux Kernel Architecture and Windows NT Architecture

(OPERATING SYSTEM CONCEPTS ABRAHAM SILBERSCHATZ)

Operating System Design And Principles

Design Goals

- We have to face several problems designing and implementing an operating system.
- The first problem in designing a system is **to define goals and specifications**
- At the highest level, the design of the system will be affected by the **choice of hardware and the type of system:** traditional desktop/laptop, mobile, distributed, or real time.
- Next, it can be divided into two basic groups: **user goals and system goals.**
- Users seek convenience, reliability, safety, and speed, while developers focus on ease of design, flexibility, reliability, and efficiency, though these goals are often vague and open to interpretation.

Operating System Design And Principles

Design Goals

- We have to face several problems designing and implementing an operating system.
- For example, the requirements for a real-time operating system for embedded systems, must have been substantially different from those for Windows Server, a large multiaccess operating system designed for enterprise applications.
- Specifying and designing an operating system is a highly creative task. There is, in short, no unique solution to the problem of defining the requirements for an operating system.

Operating System Design And Principles

Mechanisms and Policies

- Specifying and designing an operating system is a highly creative task. Policies determine what is done, while mechanisms determine how to do it.
- E.g.
    1. In Memory management, policy decides which process gets memory, such as using a "first-come, first-served" approach or prioritizing processes based on their urgency or type. The mechanism involves implementing techniques like paging or segmentation to manage memory allocation.
    2. In file management, File permissions Mechanism (e.g., read, write, execute) are checked by the system. Policy decides who has access to specific files (e.g., only the owner and administrator can modify system files).

Operating System Design And Principles

Mechanisms and Policies

- Separating policy from mechanism ensures adaptability, allowing policies to change without altering the underlying mechanisms, making systems more flexible.
- Policy decisions are crucial for allocating resources in operating system enviroment, defining what to allocate, while mechanisms define how to implement the allocation.

Operating System Design And Principles

Implementation

- Once an operating system is designed, it must be implemented. Because operating systems are collections of many programs, written by many people over a long period of time, it is difficult to make general statements about how they are implemented.
- There is diversity in language to implement and Operating System. Early operating systems were implemented in assembly language. Modern systems use higher-level languages like C or C++ , with minimal assembly for low-level tasks.
- Writing operating systems in higher-level languages enables them to run on diverse hardware platforms like embedded devices, Intel x86 systems, and ARM-based devices.
- While higher-level languages may slightly reduce speed and increase storage requirements, modern compilers often produce highly optimized code comparable to manually written assembly.

Operating System Design And Principles

Implementation

- Major performance improvements come from better algorithms and data structures rather than hand-written assembly. Only certain parts of the OS, like interrupt handlers, I/O manager, memory manager, and CPU scheduler, are critical to performance and may require focused optimization.
- Tradeoff: While high-level implementation may sacrifice some performance, it greatly enhances development speed, maintainability, and adaptability, outweighing the drawbacks in modern computing environments.